UNIVERSITY OF TECHNOLOGY, JAMAICA
COLLEGE/ FACULTY: Engineering and Computing
SCHOOL/DEPARTMENT: Computing and Information Technology

Course: Programming for Data Analytics **Course code: CIS1000**

Given: March 16, 2026 **Group Size: 4 members (2 pairs)**

Group Project Due: Sunday April 5, 2026 @ 7:00 pm

NB: The use of AI to generate a solution has serious consequences according to the University policy.

Project Description:

Create a Python application that provides users with an interactive personal finance simulation suite. Each group will consist of **four students divided into two pairs**. Each pair will be responsible for developing one simulation module, but both simulations must be integrated into a **single Python application** with one common menu system. Each group should select and develop two simulation tools of their choice, with each pair taking lead responsibility for one simulation. The application should have a common and consistent menu and user interface, and the entire project must be submitted as a single Python (.py) file.

The required programming concepts may be distributed across the two simulations. However, all features listed below must appear somewhere in the final application.

a. Random number generation (e.g., simulating market fluctuations or interest rate changes) must be used in at least one of the two simulations.

b. OOP is only required in one simulation. At least one simulation must demonstrate Object-Oriented Programming using classes, objects, and methods.

c. Strings, lists and concatenations (e.g., generating financial summaries or transaction histories)

d. Exception handling (e.g., handling invalid input amounts or negative balances)

e. The application must calculate at least four descriptive statistics across the simulations such as mean, mode, median, count, range, minimum, maximum etc.

f. User defined functions (e.g., reusable functions for calculations, formatting output, or displaying menus)

The application should store the user's simulation results and display this information along with summary statistics if the user runs multiple simulations in one sitting. The application must continue running until the user selects the exit option from the menu system. Code reuse is allowed; however, any external code or algorithms must be referenced in a docstring within the program. Trivial or overly simplistic simulations (for example, a basic interest calculator with a single input) will receive reduced marks**.** Simulations should be informative, realistic, and engaging.

Development Process

Each pair takes lead responsibility for one simulation tool, but all group members should contribute meaningfully and be able to explain and demo any part of the application. Groups should use Git and GitHub/GitLab for version control management. Access to the group's repository should be given to the lecturer/tutor for the conduct of weekly status reviews. The application must provide a common menu system and consistent user interface across both simulations. The application should be tested and run without errors.

The group should follow one consistent coding standard, and all functions and sections should be clearly documented using comments.

The final submission must include:

- one Python (.py) file containing the complete application
- the group's Git repository link.

Suggested Simulation Ideas (pairs choose any 2):

1. Compound interest & savings growth simulator
2. Stock portfolio tracker with simulated daily price changes
3. Loan repayment calculator with amortisation schedule
4. Budget planner and monthly expense analyzer
5. Retirement fund projector with inflation adjustment
6. Currency exchange simulator with historical rate trends

Grading Scheme

The final product will be graded by demo and interview during tutorial & lab times of week 12 & 13. Each group demonstration will last approximately 10 - 15 minutes, during which students must run the application and answer questions about the code. Demonstration will be done as a group; all 4 members are expected to be present and able to explain any part of the code.

Please see the rubric below:

Category	Criteria	Expectation	Marks
Functionality (35%)	Program runs successfully	Program executes without crashing and integrates both simulations in to a single application	5
	Menu system	Interactive menu allows users to select simulations and exit	5
	Required programming concepts	Proper use of functions, lists, strings, and exception handling	5
	Random number feature	One simulation correctly uses random number generation	5
	Object-Oriented Programming	Classes, objects, and methods implemented correctly	5
	Analytics calculations	At least four descriptive statistics implemented	5
	Data storage during session	Simulation results stored and summarized	5
User Interface (10%)	Menu clarity	Menu options clearly labelled	3
	Output formatting	Results displayed clearly	3
	Consistent interface	Both simulations follow same design	2
	Input validation	Invalid input handled without crashing	2
Creativity & Teamwork (15%)	Simulation realism	Financial scenario is realistic	5
	Creativity	Features go beyond minimum requirements	5
	Collaboration	Evidence all members contributed	5
Individual Contribution (40%)	Understanding of code	Student explains code during demo	10
	Contribution to simulation	Student contributed to assigned simulation	10
	Code quality	Code uses comments, readable variables, structure	10
	Problem solving	Student answers questions during demo	10

****** End of Project ******